Break, continue, nested loops and primes

A prime test built step by step, two ways to stop a loop early, skipping a pass, and loops inside loops.

THE QUESTION

How can a program decide that 97 is prime, and why does it not need to try 95 divisors?

THE INTUITION

Try the possible divisors one at a time. The first one that divides evenly proves the number is not prime, so the loop can stop there. Divisors come in pairs, so trying up to $\sqrt{n}$ is enough.

KEY TAKEAWAY

Keep a flag, leave the loop with `break` at the first divisor, and test only while `i * i <= n`. An inner loop runs all its passes for every pass of the outer one.

n → try i = 2, 3, 4, … → n % i == 0 ? → yes: not prime, break → none while i * i <= n → n is prime

1 A prime has no divisor from 2 to $n - 1$

prime.py: try every i from 2 to n - 1

```
1 n = 15
2 factor_found = False
3 i = 2
4 while i < n:
5     if n % i == 0:
6         factor_found = True
7     i = i + 1
8 print(n > 1 and not factor_found)
```

change n on line 1 and run

n	divisors of n	prints
15	1 3 5 15	False
13	1 13	True
91	1 7 13 91	False
97	1 97	True
2	1 2	True
1	1	False

green: exactly two divisors, a prime

A **divisor** of `n` divides it with no remainder: `n % i == 0` (see the variables sheet). A **prime** is a whole number above 1 whose only divisors are 1 and itself. **Trial division** tries every `i` from 2 to $n - 1$. `factor_found` is a **flag**: a bool that starts False and turns True when a divisor shows up. `not` flips a bool; `n > 1` and rules out 1, which is not prime.

2 A trace table shows each pass

`n = 15` 13 tries, prints False

i	15 % i	factor_found
2	1	False
3	0	True
4	3	True
5	0	True
6	3	True
7	1	True
8	7	True
9	6	True
10	5	True
11	4	True
12	3	True
13	2	True
14	1	True

i = 15: i < n is False, loop ends

True from i = 3 on; 11 more tries change nothing

`n = 13` 11 tries, prints True

i	13 % i	factor_found
2	1	False
3	1	False
4	1	False
5	3	False
6	1	False
7	6	False
8	5	False
9	4	False
10	3	False
11	2	False
12	1	False

i = 13: i < n is False, loop ends

no remainder is 0: the flag stays False

rose: i divides n · blue: this pass changed the flag

One row per pass, with the flag as it is after the pass (trace tables: see the while loops sheet). `15 % 3` is 0, so 15 is not prime, yet nothing tells the loop to stop. For 13 no remainder is 0, so the flag stays False. Nothing ever sets the flag back to False.

3 Stop at the first divisor: a flag in the test, or `break`

A · the flag in the test (notebook)

```
while i < n and not factor_found:
    if n % i == 0:
        factor_found = True
    i = i + 1
```

B · break

```
while i < n:
    if n % i == 0:
        factor_found = True
        break
    i = i + 1
```

		passes (n % i tried)			i after the loop	
n	prints	every i	A	B	A	B
13	True	11	11	11	13	13
15	False	13	2	2	4	3
91	False	89	6	6	8	7
97	True	95	95	95	97	97

A and B: same answers, same passes. With break, i stays on the divisor.

Once the flag is True the answer is known. A adds `and not factor_found` to the test (the notebook writes `factor_found != True`: the same). B uses `break`, which ends the loop at once; Python goes on after the loop. For a prime neither saves a pass.

4 `break` jumps out and skips the rest of the pass

prime.py with break, n = 25

```
1 n = 25
2 factor_found = False
3 i = 2
4 while i < n:
5     if n % i == 0:
6         factor_found = True
7         break
8     i = i + 1
9 print(n > 1 and not factor_found)
```

lines that ran; i in each pass



blue: ran · rose: break ran and jumped out · green: the line after the loop

Python's **tracer** (`sys.settrace`) recorded the lines that ran. Passes 1 to 3 run the test, the `if` and the update. In pass 4, 5 divides 25: line 6 sets the flag and `break` jumps to line 9, so the update and the test do not run again. `break` works only inside a loop.

5 Stop at $\sqrt{n}$: divisors come in pairs

36 as a rectangle of dots, every way: one side is always 6 or less



then 9 × 4, 12 × 3 and 18 × 2: the same pairs again

Once i * i > n with no divisor found, there is none left to find.

	divisions tried (n % i), stopping at the first divisor			
n	i < n	i * i <= n	odd i only	prime?
97	95	8	5	True
91	6	6	4	False
1,000,003	1,000,001	999	500	True

counted from the running code; odd i only counts the n % 2 test too

If $n = a \times b$ with $a \le b$, then $a \times a \le n$: the smaller divisor is at most $\sqrt{n}$. So `while i * i <= n:` is enough, with ints only and no float error; the notebook's `i < math.ceil(math.sqrt(n)) + 1` tries at most one value more. For the prime 1,000,003 that is 999 tries instead of 1,000,001; testing 2 once and then only odd `i`, the notebook's last version, halves it again. Keep the =: with `i * i < n`, 25 comes out prime.

Skipping a pass, and loops inside loops

continue traced line by line, nested loops traced pass by pass, a triangle, all pairs, and every prime below 50.

6 continue skips the rest of this pass

contine-check.py

```
1 i = 0
2 s = 0
3 while i < 10:
4     i = i + 1
5     if i % 2 == 0:
6         continue
7     s = s + i
8 print(s)
```

i in each pass (Python's tracer)



amber: continue ran, so line 7 was skipped · green: s after the pass

continue ends the current pass early and jumps back to the while test. This script adds up the odd numbers to 10: on even passes line 6 runs and line 7 is skipped, so s ends at 25 (1 + 3 + 5 + 7 + 9). The update comes first, so continue never skips it.

7 A loop inside a loop: the inner one runs in full each time

table.py

```
1 row = 1
2 while row <= 3:
3     column = 1
4     while column <= 3:
5         print(row * column, end=" ")
6         column = column + 1
7     print()
8     row = row + 1
```

output

```
1 2 3
2 4 6
3 6 9
```

row	column	prints
1	1	1
1	2	2
1	3	3
1	4	print(): new line
2	1	2
2	2	4
2	3	6
2	4	print(): new line
3	1	3
3	2	6
3	3	9
3	4	print(): new line

column: 1, 2, 3 (4 ends it), each row

A **nested loop** sits inside the body of another. For each pass of the **outer** loop (row), the **inner** loop (column) restarts at 1 and runs all its passes: $3 \times 3 = 9$ prints. end=" " keeps a row on one line; print() ends it.

8 The inner loop can depend on the outer variable

triangle: col <= row

```
1 row = 1
2 while row <= 4:
3     col = 1
4     while col <= row:
5         print("*", end="")
6         col = col + 1
7     print()
8     row = row + 1
```

```
*      row 1: 1 star
**     row 2: 2 stars
***    row 3: 3 stars
****   row 4: 4 stars
```

pairs: j from i + 1

```
1 i = 1
2 while i <= 4:
3     j = i + 1
4     while j <= 4:
5         print(i, j)
6         j = j + 1
7     i = i + 1
```

prints

```
1 2   2 3
1 3   2 4
1 4   3 4
```



number: the order it was printed in

With col <= row as the inner test, row 1 gets 1 star and row 4 gets 4 (10 in all). For **pairs** i < j the inner loop starts at j = i + 1: no pair twice, no number with itself. 4 numbers give 6 pairs, $4 \times 3/2$.

9 Every prime below 50: a prime test inside a loop

primes.py

```
1 n = 2
2 while n < 50:
3     is_prime = True
4     i = 2
5     while i * i <= n:
6         if n % i == 0:
7             is_prime = False
8             break
9         i = i + 1
10    if is_prime:
11        print(n, end=" ")
12    n = n + 1
```

n from 1 to 49; small: the i where break ran

1	2	3	4 2	5	6 2	7	8 2	9 3	10 2
11	12 2	13	14 2	15 3	16 2	17	18 2	19	20 2
21 3	22 2	23	24 2	25 5	26 2	27 3	28 2	29	30 2
31	32 2	33 3	34 2	35 5	36 2	37	38 2	39 3	40 2
41	42 2	43	44 2	45 3	46 2	47	48 2	49 7	

prints: 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 — 15 primes · 94 divisions in all

The outer loop walks n from 2 to 49; the inner loop is the $\sqrt{n}$ test, with the flag is_prime set back to True for each n. break leaves only the inner loop, and the outer one moves on to the next n.

10 Common mistakes: what you wrote, what happens, the fix

you wrote	what happens	why, and the fix
`if i % 2 == 0: continue` `i = i + 1`	never ends; Ctrl+C gives KeyboardInterrupt	i stays 0: continue skips the update. Put i = i + 1 first in the body; it prints 1 3 5 7 9.
`else:` `    factor_found = False`	no error; prints True for 15	the else sets the flag back on a later pass (i = 14). Only ever set the flag to True in the loop.
`column = 1` `while row <= 3:`	no error; prints 1 2 3 only	column is 4 after row 1, so the inner loop never runs again. Set column = 1 inside the outer loop.
`if n % 3 == 0: break`	SyntaxError: 'break' outside loop	use break and continue only inside a loop.
`print(not factor_found)`	no error; prints True for 1	for 1 the loop runs zero times, so the flag stays False. Print n > 1 and not factor_found.

11 Try it (answers in green)

1. i goes 1 to 6; if i == 4: break comes before print(i). Prints?
1 2 3: the loop ends at 4.

2. The same with continue in place of break?
1 2 3 5 6: only 4 is skipped.

3. How many n % i does i * i <= n try for 121?
10: i = 2 to 11 (11×11).

4. row 1 to 3, inner col 1 to 4: how many inner passes?
12: 4 for each of 3 rows.

5. How many pairs i < j can you make from 1 to 5?
10: 4 + 3 + 2 + 1.

6. Is 221 prime? At which i does the $\sqrt{n}$ test stop?
No: break at 13 (13×17).

break leaves the loop · continue goes back to the test · stop once i * i > n · inner loops restart each pass

Source: the notebooks on primes, break, continue and nested loops, and contine-check.py, nipunbatra.github.io/comp22. Outputs, traces and counts come from running the code with Python 3.11.11; the endless loop was stopped with Ctrl+C. Next: the lists sheet.